Exercise 1 - Hints and solutions

Try each task yourself first, and look at the solution only when you are stuck. Remember: there is often more than one correct solution!

Part A

A1. `print` vs `return`

Hint: Keep the loop as it is, and replace `print(count)` with `return count` and change the name of the function.

Solution:

In [ ]:
```
def gc_count(seq):
    count = 0
    for base in seq:
        if base == 'G' or base == 'C':
            count += 1
    return count

total = gc_count('ATGC') + gc_count('GGCC')
print(total)
```

A2. Return more than one value

Hint: Use two counters and two if statements with **or** in the condition. One for `A` or `T` and one for `G` or `C`. Return both, separated by a comma:

```
return at, gc
```

Solution:

In [ ]:
```python
def count_at_gc(seq):
    at = 0
    gc = 0
    for base in seq:
        if base == 'A' or base == 'T':
            at += 1
        if base == 'G' or base == 'C':
            gc += 1
    return at, gc

at, gc = count_at_gc('ATGCGC')
print(at, gc)
```

A3. Positional and keyword arguments

Hint: With positional arguments the **order** matters. With keyword arguments you write `name=value`, and the order does not matter.

Solution:

In [ ]:
```python
def describe_gene(name, chrom, start):
    print(f"{name} is on {chrom} and starts at position {start}")

# 1. positional
describe_gene('BRCA1', 'chr17', 43044295)

# 2. keyword, different order
describe_gene(start=43044295, name='BRCA1', chrom='chr17')

# 3. mixed
describe_gene('BRCA1', start=43044295, chrom='chr17')
```

`describe_gene(name='BRCA1', 'chr17', 43044295)` gives:

```
SyntaxError: positional argument follows keyword argument
```

Positional arguments must always come **in front of** keyword arguments.

A4. Default arguments

Hint: A default value is written in the definition: `def count_base(seq, base='C'):`. Inside the loop, compare each letter with `base` instead of `'C'`.

Solution:

In [ ]:
```python
def count_base(seq, base='C'):
    count = 0
    for letter in seq:
        if letter == base:
            count += 1
```

```
    return count

print(count_base('ACCG'))
print(count_base('ACCG', base='G'))
```

Part B - IMDB movie picker, step by step

The outline of the function:

- read the file, line by line
- for each movie, check the criteria (`genre` , `year` , `rating_min`). For each criterion:
  - check if it is **active**: the user has given a value, so it is **not** `None`
  - if it is active, check if the movie matches it
- if the movie matches all active criteria, **return** its title
- if no movie matched, **return** `None`

Step 1 - Read the file

A line in the file looks like this:

```
126807|   8.5|1957|5280|https://....jpg|Drama,War|Paths of Glory
```

The values are separated by `|` , so we split each line at `|` :

```
fields = line.split('|')
```

which gives

```
['126807', '   8.5', '1957', '5280', 'https://....jpg', 'Drama,War', 'Paths of Glory\n']
```

Now save the values we need in variables. Think about the **type** of each value:

- `rating` is at position `1` . We will compare it with `<` , so it must be a number: `float(fields[1])`
- the year is at position `2` : `int(fields[2])` . We call it `m_year` (movie year), because the name `year` will be used for the function argument
- the genres are at position `5` . A movie can have many genres, so we split them at `,` to get a list: `fields[5].strip().split(',')`
- the title is at position `6` . Use `.strip()` to remove the newline at the end: `fields[6].strip()`

Open the file as usual with `with open(...) as fh:` , and loop over its lines with `for line in fh:` . Remember to skip the header line, which starts with `#` .

Let's first check that reading works, by printing the first 3 titles:

In [ ]:
```
imdb = "../../downloads/250.imdb" # Add your relative path of the file here
```

In [ ]:
```
count = 0
with open(imdb, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            fields = line.split('|')
            rating = float(fields[1])
            m_year = int(fields[2])
            genres = fields[5].strip().split(',')
            title = fields[6].strip()
            print(title, m_year, rating, genres)
            count += 1
            if count == 3:
                break
```

Step 2 - A function with one criterion: `genre`

Put the code inside a function. The genre is active if it is not `None`. Check if it is a member of the list `genres` with `in`.

As soon as we find a matching movie we use `return`. This stops the loop and the whole function.

The file is closed automatically when we leave the `with` block, also when we leave it with `return`.

In [ ]:
```
def pick_movie(genre=None):
    with open(imdb, 'r', encoding='utf-8') as fh:
        for line in fh:
            if not line.startswith('#'):
                fields = line.split('|')
                rating = float(fields[1])
                m_year = int(fields[2])
                genres = fields[5].strip().split(',')
                title = fields[6].strip()

                if genre is not None and genre in genres:
                    return title

pick_movie(genre="Drama")
```

Step 3 - Add `year` and `rating_min`

With more criteria, it is easier to start by assuming that the movie is ok, and then check each criterion. If one criterion does not match, the movie is not ok. We store this in the variable `movie_ok`:

```
movie_ok = True                    # the movie is ok, until we find a
problem

if genre is not None and genre not in genres:
    movie_ok = False               # active, but does not match
```

Do the same for the other criteria:

- `year` : the movie's year must be **equal** to `year`
- `rating_min` : the movie's rating must **not be less than** `rating_min`

Step 4 - When no movie matches

If the loop finishes without finding a movie, we reach the end of the function. A function without `return` returns `None` anyway, but writing `return None` makes it clear to anyone who reads the code.

The complete function

In [ ]:
```
def pick_movie(genre=None, year=None, rating_min=None):
    with open(imdb, 'r', encoding='utf-8') as fh:
        for line in fh:
            if line.startswith('#'):
                continue
            fields = line.split('|')
            rating = float(fields[1])
            m_year = int(fields[2])
            genres = fields[5].strip().split(',')
            title = fields[6].strip()

            # Assume the movie is ok, and reject it if a criterion does not
            movie_ok = True
            if genre is not None and genre not in genres:
                movie_ok = False
            if year is not None and year != m_year:
                movie_ok = False
            if rating_min is not None and rating < rating_min:
                movie_ok = False

            # All criteria ok: return the title (this also stops the loop)
            if movie_ok:
                return title

    # No movie matched
    return None
```

Let's test it!

In [ ]:
```
pick_movie(genre="Drama")
```

In [ ]:
```
pick_movie(year=2009, genre="Mystery")
```

In [ ]:
```
pick_movie(rating_min=9)
```

In [ ]:
```
print(pick_movie(genre="Horror", year=2001))
```

Because the function **returns** the title, we can use it in our code:

In [ ]:
```
movie = pick_movie(year=2001)
print(f"Tonight we watch: {movie}")
```